

Máquina de Bolas

Madina inventou uma máquina de bolas para entreter os participantes da IOI. A estrutura interna da máquina é a seguinte:

- A máquina consiste em N **nós**, indexados de 0 a $N - 1$. O nó $N - 1$ é chamado de **raiz**. Para cada nó u com $0 \leq u < N - 1$, o **pai** do nó u é um nó $P[u]$ com $P[u] > u$, e o nó u é um **filho** de $P[u]$. A raiz não tem pai. Um nó sem filhos é chamado de **folha**.

Você **não** conhece N nem o pai $P[u]$ de qualquer nó u . Em vez disso, Madina lhe informa M , o número de folhas, e que os índices das folhas são $0, 1, \dots, M - 1$. Sua tarefa é determinar a estrutura interna da máquina operando-a.

Cada nó da máquina pode armazenar no máximo uma **bola**, e cada bola tem um **valor** que é um número inteiro não negativo. Dizemos que um nó tem valor x se ele armazena uma bola com valor x . Um nó está **ocupado** se contiver uma bola; caso contrário, está **livre**. Inicialmente, todos os nós estão livres.

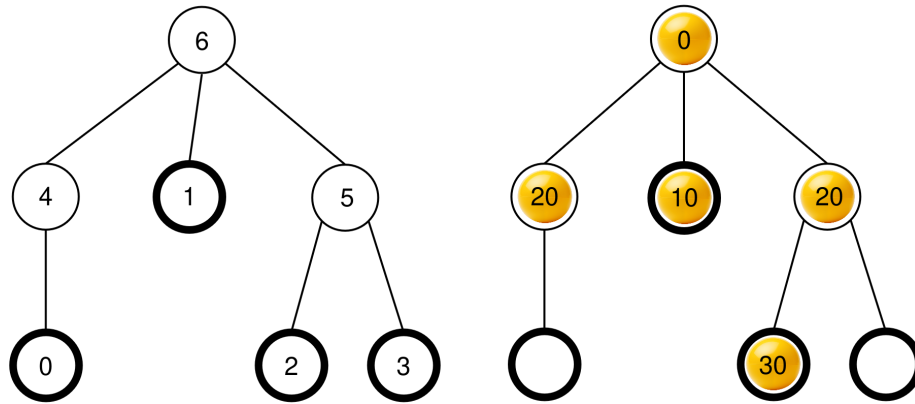
Você pode realizar dois tipos de operações:

- insert**(U, X): escreva o valor X em uma nova bola e tente inseri-la na folha U .
 - Se a folha U estiver ocupada, a operação retorna `false` sem inserir a bola.
 - Se a folha U estiver livre, a bola é colocada no nó U . Então, a bola se move repetidamente para o pai do seu nó atual, contanto que esse pai esteja livre. O movimento para quando a bola atinge a raiz ou um nó cujo pai está ocupado. A operação retorna `true`.
- collect**(**:**): se a raiz estiver livre (o que significa que a máquina está vazia), `collect` retorna um vetor vazio. Caso contrário, o procedimento recursivo `traverse` descrito abaixo coleta os valores de todas as bolas que estão atualmente na máquina em um vetor S . Inicialmente, o vetor S está vazio e `traverse`($N - 1$) é chamado.

```
traverse(u):  
    Adicione o valor da bola no nó u ao final de S  
    Seja c[u] a lista dos filhos ocupados de u  
    Ordene c[u] em ordem não decrescente de acordo com os valores das bolas  
    nesses nós (se vários nós tiverem o mesmo valor, podem aparecer em  
    qualquer ordem)  
    para cada v em c[u]:  
        traverse(v)
```

Após a conclusão deste procedimento, **todas as bolas são removidas** da máquina e `collect` retorna S .

Por exemplo, considere uma máquina com $N = 7$ nós e um vetor pai $P = [4, 6, 5, 5, 6, 6]$. A imagem à esquerda mostra a máquina vazia com os índices dos nós, enquanto a imagem à direita mostra uma possível configuração de bolas após uma sequência de operações de `insert` (essa sequência é fornecida na seção Exemplo).



Quando `collect()` é chamado, a raiz (nó 6) está ocupada, então S é inicializado como $S = []$ e `traverse(6)` é chamado.

`traverse(6)`: o nó 6 tem valor 0; adicionando-o a S resulta em $S = [0]$. Os filhos do nó 6 são 4, 1 e 5, todos ocupados. Seus valores são 20, 10 e 20, respectivamente. Ordenando em ordem não decrescente, obtemos $c[6] = [1, 5, 4]$ (observe que $c[6] = [1, 4, 5]$ também seria válido, já que os nós 4 e 5 têm o mesmo valor). O procedimento então chama `traverse` em cada nó de $c[6]$ nessa ordem.

- **`traverse(1)`**: o nó 1 tem o valor 10; adicionando-o a S resulta em $S = [0, 10]$. O nó 1 não tem filhos ocupados, então esta chamada termina.
- **`traverse(5)`**: o nó 5 tem valor 20; adicionando-o a S resulta em $S = [0, 10, 20]$. O nó 5 tem um filho ocupado, o nó 2, então $c[5] = [2]$ e o procedimento chama `traverse(2)`.
 - **`traverse(2)`**: o nó 2 tem o valor 30; adicionando-o a S resulta em $S = [0, 10, 20, 30]$. O nó 2 não tem filhos ocupados, então esta chamada termina.
 - A execução retorna para `traverse(5)` que processou todos os filhos em $c[5]$, então sua chamada termina.
- **`traverse(4)`**: o nó 4 tem o valor 20; adicionando-o a S resulta em $S = [0, 10, 20, 30, 20]$. O nó 4 não tem filhos ocupados, então esta chamada termina.

Todas as chamadas foram concluídas e não há mais nós para processar. Finalmente, todas as bolas são removidas da máquina e `collect` retorna o vetor $S = [0, 10, 20, 30, 20]$.

Sua tarefa é determinar o número de nós N e reportar um vetor de pais $R = [R[0], R[1], \dots, R[N-2]]$ que descreva a estrutura da máquina, mas com uma rotulação potencialmente diferente para os nós

que não são folhas nem a raiz. Formalmente, um vetor reportado R é considerado correto se for possível atribuir um rótulo único $L[u]$ com $0 \leq L[u] < N$ a cada nó u da máquina, de forma que:

- $L[u] = u$ para todo $0 \leq u < M$ e $u = N - 1$, e
- $L[P[u]] = R[L[u]]$ para todo $0 \leq u < N - 1$.

Não é necessário que $R[u] > u$. A máquina **deve estar vazia** quando você reportar sua solução.

Seja K o número de operações `collect` realizadas e B o valor máximo escrito em qualquer bola em todas as chamadas `insert`. Seja $C = K + B$. O valor C **não deve exceder** 1000. Sua pontuação em algumas subtarefas depende do valor de C .

Detalhes de Implementação

Você deve implementar o seguinte procedimento.

```
std::vector<int> find_structure(int M)
```

- M : o número de folhas na máquina.
- O procedimento é chamado exatamente uma vez para cada caso de teste.
- O procedimento deve retornar um vetor $R = [R[0], R[1], \dots, R[N - 2]]$ de comprimento $N - 1$, um vetor correto de pais.

Para interagir com a máquina, seu procedimento pode chamar os dois procedimentos a seguir.

O primeiro procedimento é:

```
bool insert(int U, int X)
```

- U : o índice da folha onde a bola deve ser inserida. A condição $0 \leq U < M$ deve ser satisfeita.
- X : o valor inteiro da bola. A condição $0 \leq X \leq 1000$ deve ser satisfeita.
- Este procedimento retorna `true` se a bola foi inserida com sucesso, ou `false` se a folha U já estava ocupada.
- Este procedimento pode ser chamado no máximo 500 000 vezes.

O segundo procedimento é:

```
std::vector<int> collect()
```

- Recolhe os valores de todas as bolas inseridas, esvazia a máquina e retorna o vetor recolhido S .

Se em algum momento durante a execução do seu programa o valor de C exceder 1000, sua solução receberá o veredicto `Output isn't correct: Too many resources used`.

A estrutura da máquina é definida antes da chamada de `find_structure`.

Restrições

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Subtarefas

Subtarefa	Pontuação	Restrições adicionais
1	5	A raiz tem exatamente M filhos.
2	10	$M \leq 3$
3	25	$N \leq 200$, $M \leq 45$
4	60	Sem restrições adicionais.

Nas subtarefas 3 e 4, sua pontuação depende do valor de C da seguinte forma.

Subtarefa 3 (25 pontos)

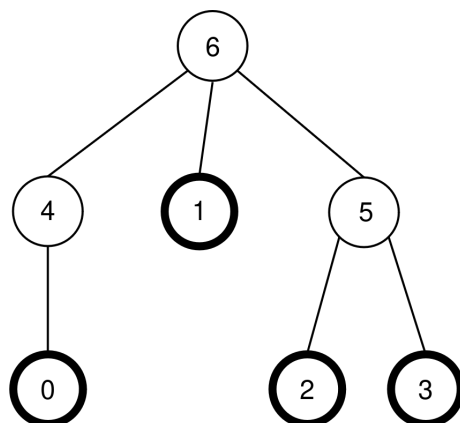
Limites	Pontuação
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Subtarefa 4 (60 pontos)

Limites	Pontuação
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Exemplo

Considere a mesma máquina da descrição da tarefa, portanto $N = 7$, $M = 4$ e $P = [4, 6, 5, 5, 6, 6]$. A máquina é mostrada novamente na figura a seguir:



O corretor faz a seguinte chamada:

```
find_structure(4)
```

O procedimento pode realizar a seguinte sequência de chamadas:

1. **insert(0, 0)**: a folha 0 está livre, então uma bola com valor 0 é colocada na folha 0. O nó $P[0] = 4$ está livre, então a bola se move para o nó 4. O nó $P[4] = 6$ está livre, então a bola se move para o nó 6. Como o nó 6 é a raiz, o movimento para. A chamada retorna `true`.
2. **insert(3, 20)**: a folha 3 está livre, então uma bola com valor 20 é colocada na folha 3. O nó $P[3] = 5$ está livre, então a bola se move para o nó 5. Como $P[5] = 6$ está ocupado, o movimento para. A chamada retorna `true`.
3. **insert(1, 10)**: a folha 1 está livre, então uma bola com valor 10 é colocada na folha 1. Como $P[1] = 6$ está ocupado, a bola permanece no nó 1. A chamada retorna `true`.
4. **insert(2, 30)**: a folha 2 está livre, então uma bola com valor 30 é colocada na folha 2. Como $P[2] = 5$ está ocupado, a bola permanece no nó 2. A chamada retorna `true`.
5. **insert(1, 25)**: a folha 1 está ocupada, então a bola não é colocada na folha 1. A chamada retorna `false`.
6. **insert(0, 20)**: a folha 0 está livre, então uma bola com valor 20 é colocada na folha 0. O nó $P[0] = 4$ está livre, então a bola se move para o nó 4. Como $P[4] = 6$ está ocupado, o movimento para. A chamada retorna `true`.

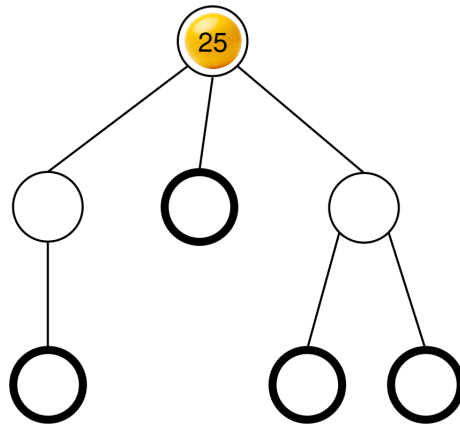
A configuração resultante das bolas já havia sido exibida na descrição da tarefa.

Em seguida, o procedimento pode chamar:

```
collect()
```

A execução dessa chamada já foi explicada na descrição da tarefa, portanto, essa chamada retorna $S = [0, 10, 20, 30, 20]$. Depois disso, a máquina fica vazia novamente.

Em seguida, o procedimento pode chamar **insert(2, 25)**. A folha 2 está livre, então uma bola com valor 25 é colocada na folha 0. Essa bola então se move para o nó 5 e, em seguida, para o nó 6, onde para. A chamada retorna `true`. A configuração resultante das bolas é mostrada na figura a seguir:



Finalmente, o procedimento pode chamar `collect()` que retorna $S = [25]$ e esvazia a máquina.

Existem dois vetores pais corretos que `find_structure` pode retornar: $R = P = [4, 6, 5, 5, 6, 6]$ (que corresponde à rotulação $L = [0, 1, 2, 3, 4, 5, 6]$), e $R = [5, 6, 4, 4, 6, 6]$ (que corresponde à rotulação $L = [0, 1, 2, 3, 5, 4, 6]$). Neste exemplo, $K = 2$ e $B = 30$, então $C = 32$.

Corretor exemplo

Formato de entrada:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Formato de saída:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Aqui, Q é o comprimento da matriz R retornada por `find_structure`.